University College of Applied Sciences (UCAS)

Department of Computer Engineering

Course: Computer Architecture

Locality of Different Programs Using SMPCache 2.0

Prepared by:
Student Name: Haya Abo Said
Student ID: 220230264
Student Name: Ilan Abu Sharekh
Student ID: 220230551
Student Name: Mariam Awad Allah
Student ID: 220240512

Submitted to:
Dr. Mohammed A. Matar
Assistant Professor

Semester: Second Semester — Date: July 13, 2026

Contents

1 Project Selection

Select the project assigned or chosen from the SMPCache manual (Check the applicable box):

- **Project 1: Locality of Different Programs**

  Project 2: Influence of the Cache Size

  Project 3: Influence of the Block Size

  Project 4: Influence of the Block Size for Different Cache Sizes

  Project 5: Influence of the Mapping for Different Cache Sizes

  Project 6: Influence of the Replacement Policy

2 Architectural Configuration

Table 1: SMPCache Simulator Environment Settings

Architectural Parameter	**Configured Value**
Number of Processors (SMP)	1
Cache Coherence Protocol	MESI
Bus Arbitration Scheme	Random
Word Width (Bits)	16
Block Size (Words / Bytes)	16 Words / 32 Bytes
Main Memory Size (Blocks / KB)	8192 Blocks / 256 KB
Cache Size (Blocks / KB)	128 Blocks / 4 KB
Mapping Scheme	Fully-Associative
Replacement Policy	LRU

3 Experimental Results

3.1 Data Table

Each memory trace was executed independently using the same fixed architectural configuration. The global cache statistics were recorded for each program because the simulated system contains only one processor.

Table 2: Cache Performance Results for the Memory Traces

Trace	Total References	Hits	Misses	Hit Rate (%)	Miss Rate (%)	Replacements
Hydro (Floating point)	2127	1812	315	85.19	14.81	187
Nasa7 (Floating point)	1855	1570	285	84.636	15.364	157
Cexp (Integer)	20000	19852	148	99.26	0.74	20
Mdljd (Floating point)	20000	17044	2956	85.22	14.78	2828
Ear (Floating point)	5308	4772	536	89.9	10.09	408
Comp (Integer)	2524	2072	452	82.09	17.91	324
Wave (Floating point)	3427	2828	599	82.521	17.479	471
Swm (Floating point)	20000	16074	3926	80.37	19.63	3798
UComp (Integer)	2733	2261	472	82.73	17.27	344

3.2 Graphical Analysis

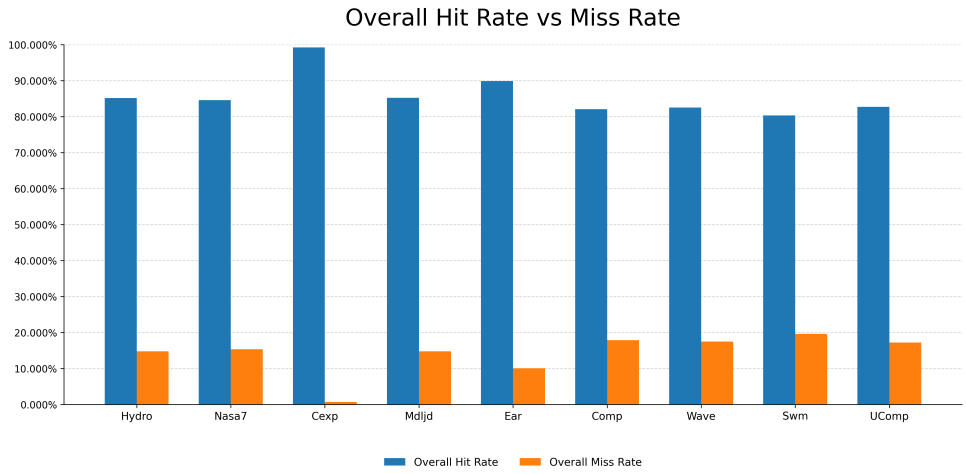

Figure 1: Overall Cache Hit and Miss Rates Across the Memory Traces

4 Analysis and Discussion

4.1 Question 1

Do all the programs have the same locality grade?

Answer: No, the programs do not exhibit the same degree of locality. Under the same cache configuration, the experimental results show different cache miss rates for the tested memory traces, indicating that the programs follow different memory-access patterns. Programs that repeatedly access recently used data exhibit better temporal locality, while programs that access nearby memory locations exhibit better spatial locality. These access patterns generally result in more cache hits and fewer cache misses. In contrast, scattered or irregular memory accesses tend to exhibit poorer locality and higher miss rates.

4.2 Question 2

Which program has the best locality, and which one has the worst?

Answer: The Cexp program exhibits the best locality because it has the highest cache hit rate of 99.26% and the lowest miss rate of 0.74%. This indicates that it frequently reuses recently accessed data or accesses nearby memory locations, demonstrating strong temporal and spatial locality. In contrast, the Swm program exhibits the worst locality because it has the lowest cache hit rate of 80.37% and the highest miss rate of 19.63%. It also produces a large number of cache replacements, suggesting that its memory accesses are less regular or more widely scattered.

4.3 Question 3

Do you think that designing memory systems to exploit the locality of the kinds of programs most commonly executed on a system can increase system performance? Why?

Answer: Yes. Designing a memory system to exploit the locality exhibited by the programs most commonly executed on the system can significantly improve performance. Temporal locality can be exploited by keeping recently used instructions and data in cache memory, while spatial locality can be exploited through appropriate cache-block sizes and prefetching nearby memory locations. These techniques increase the cache hit rate, reduce the average memory-access time, and decrease the time the processor spends waiting for data. Consequently, program execution becomes faster and overall system performance improves. However, the memory system should not be overly specialized, because programs with different memory-access patterns may receive less benefit or may experience additional cache misses.

5 Conclusion & Key Takeaways

The experimental results demonstrate that program performance is strongly influenced by memory-access locality. Although all traces were executed using the same cache configuration, they produced different hit rates, miss rates, and numbers of cache replacements because each program follows a different memory-access pattern. The Cexp (Integer) trace exhibited the best locality, achieving the highest hit rate of 99.26% and the lowest miss rate of 0.74%. In contrast, the Swm (Floating point) trace showed the worst locality, with the lowest hit rate of 80.37% and the highest miss rate of 19.63%.

A high cache hit rate allows most memory requests to be served quickly from the cache, reducing the average memory-access time and the time the processor spends waiting for data. Conversely, a high miss rate causes more accesses to slower levels of the memory hierarchy and may increase cache replacements, which can reduce overall execution efficiency. Therefore, memory systems that effectively exploit temporal and spatial locality can significantly improve system performance. The results also confirm that cache performance depends not only on the cache design, but also on the access behavior of the executed program.